Autor:
Ronie Martínez

Formatos de ficheros y acceso con Python

Introducción

Los ficheros se crean para almacenar información a través de aplicaciones informáticas específicas, como hojas de cálculo o documentos creados por un editor de textos. Un fichero es un conjunto de datos relacionados entre sí que se almacena permanentemente en dispositivos como discos duros, memorias flash, etc. y que, hasta cierto punto, pueden tratarse como una unidad. En este contexto, "permanente" significa que estos datos permanecen en el medio en que se almacenan (medios magnéticos o de otro tipo) y continúan existiendo incluso después de que el programa que los creó deja de ejecutarse, salvo fallos catastróficos, o hasta que sean borrados a propósito. Esto los distingue de los datos provisionalmente almacenados en la memoria RAM (datos volátiles de la memoria de la computadora), que no sobreviven al programa que los crea y mucho menos a la desconexión de la computadora o sistema. El contenido y la estructura de un fichero responde a un criterio de diseño libre y arbitrario, elegido por el desarrollador de una aplicación (Trespaderne & Echevarría, 2023).

La manipulación de datos y la interacción con bases de datos son fundamentales en el proceso de desarrollo de software y análisis de datos. Python, con su amplio conjunto de librerías, ofrece diversas herramientas para importar y manejar datos desde una variedad de formatos, como bases de datos (db), ficheros de tipo CSV, XML, JSON, entre otros.

Una de las formas más comunes de interactuar con bases de datos en Python es mediante el uso de la librería sqlite3, que permite conectarse a bases de datos SQLite y realizar operaciones como consultas, inserciones, actualizaciones y eliminaciones de datos. Además, existen otras librerías como psycopg2 y mysql-connector-python que permiten conectarse a bases de datos PostgreSQL y MySQL, respectivamente. Existe también la opción para interactuar con bases de datos en Python mediante la librería Pandas, que permite leer y escribir datos en bases de datos SQL mediante la función **read_sql()** y **to_sql()**, respectivamente. Además, pandas también permite realizar operaciones de análisis de datos en los datos obtenidos de la base de datos.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Python también ofrece funciones y librerías para importar datos con diversos formatos, por ejemplo, **csv.reader()** o **pandas.read_csv()** que implementa clases para leer y escribir datos tabulares en formato CSV, **xml.etree.ElementTree** para archivos XML y **json.load()** para archivos JSON que de igual manera permiten leer y escribir datos en estos formatos de manera sencilla y eficiente. Las librerías como Pandas permiten realizar operaciones complejas en los datos, como filtrar, agrupar, y transformar. Además, las características de listas y diccionarios en Python facilitan la manipulación de datos en estructuras más básicas. Adicionalmente, Python proporciona métodos para guardar datos manipulados de nuevo en diferentes formatos. Por ejemplo, **pandas.to_csv()** para guardar un DataFrame en un archivo CSV, y **json.dump()** para escribir datos en un archivo JSON, etc.

Las librerías de Python facilitan la manipulación de datos, tanto estructurados como no estructurados. Una ventaja de utilizar Python para interactuar con bases de datos es justamente esta gran cantidad de librerías disponibles que facilitan el proceso. Además, Python es un lenguaje de programación muy popular y cuenta con una gran comunidad de desarrolladores que comparten sus conocimientos y crean nuevas herramientas y librerías constantemente.

Librerías

SQLite3

SQLite3 es un módulo integrado (librería) en Python (basado en C) que proporciona un motor de base de datos relacional liviano basada en disco (disk-based database) que no requiere un proceso de servidor separado. Permite acceder a la base de datos utilizando una variante no estándar del lenguaje de consulta SQL permitiendo usar comandos como: **CREATE TABLE**, **INSERT**, **SELECT**, **UPDATE**, and **DELETE**. Algunas aplicaciones pueden usar SQLite para almacenamiento interno. También es posible prototipar una aplicación usando SQLite y luego transferir el código a una base de datos más grande como PostgreSQL u Oracle. Esto lo hace especialmente adecuado para proyectos más pequeños y aplicaciones embebidas.

Aunque en la mayoría de las distribuciones de Python **sqlite3** está incluido y no debe ser necesaria la instalación, el comando para hacerlo es el siguiente:

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

!pip install db-sqlite3

import sqlite3

Para comenzar a trabajar con SQLite en Python, es necesario establecer esta conexión a la base de datos. La conexión se la realiza mediante el método **sqlite3.connect()** y se lo debe asignar a un objeto:

db = sqlite3.connect("my_database.db")

Se pueden ejecutar comandos SQL utilizando el método **execute()**, ya sea directamente desde el objeto o mediante un objeto cursor (utilizado para ejecutar comandos SQL y recuperar resultados):

db.execute("create table usuarios(id INTEGER PRIMARY KEY, nombre TEXT, edad INTEGER)")

o

cursor = db.cursor()

cursor.execute('''CREATE TABLE IF NOT EXISTS usuarios (id INTEGER PRIMARY KEY, nombre TEXT, edad INTEGER)''')

No es necesario declarar el tipo de datos de cada columna (flexible typing), pero es una buena práctica hacerlo. Se pueden consultar las tablas disponibles/ creadas mediante el siguiente comando:

res = db.execute("SELECT name FROM sqlite_master")

res.fetchone()

Para insertar datos en la base de datos se utiliza la sentencia SQL **insert**:

cursor.execute("INSERT INTO usuarios (nombre, edad) VALUES (?, ?)", ('Juan', 25))

Para recuperar datos se utiliza la sentencia SQL **select**:

cursor.execute("SELECT * FROM usuarios")

usuarios = cursor.fetchall()

for usuario in usuarios:

** print(usuario)**

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Después de realizar los cambios en la base de datos, se debe hacer un commit para aplicar estos cambios, y posteriormente cerrar la conexión:

```
db.commit()

db.close()
```

Además de las operaciones de selección e inserción, se pueden realizar operaciones de actualización y eliminación utilizando las sentencias **update** y **delete**.

SQLite en Python también admite transacciones para agrupar varias operaciones en una unidad atómica:

```
with db:

  db.execute("INSERT INTO usuarios (nombre, edad) VALUES (?, ?)", ('Ana', 30))

  db.execute("UPDATE usuarios SET edad = 31 WHERE nombre = 'Ana'")
```

De igual manera se pueden agregar múltiples filas de datos:

```
cursor.execute("""

   INSERT INTO usuarios VALUES

      ('Ana', 25),

      ('Grace', 21)

""")
```

También se puede utilizar listas de Python:

```
data = [("Name1", 35),

      ("Name2", 20),

      ("Name3", 31)]

cursor.executemany("INSERT INTO usuarios VALUES(?, ?, ?)", data)
```

Se debe considerar que la sentencia **insert** utilizada dentro del método **execute()**, implícitamente abre una transacción, la cual necesita ser confirmada mediante **commit()** para que los cambios sean guardados en la base de datos.

Es importante considerar el manejo de errores al interactuar con bases de datos para gestionar posibles problemas durante la ejecución.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Una declaración SQL puede utilizar uno de dos tipos de marcadores de posición (placeholders): signos de interrogación (estilo qmark) o marcadores de posición con nombre (estilo con nombre). Para el estilo qmark, los parámetros deben ser una secuencia cuya longitud debe coincidir con el número de marcadores de posición, o saltará un **ProgrammingError**. Para el estilo con nombre, los parámetros deben ser una instancia de un **dict** (o una subclase), que debe contener claves para todos los parámetros con nombre; cualquier elemento adicional se ignora. Aquí hay un ejemplo de ambos estilos:

```
con = sqlite3.connect(":memory:")
cur = con.execute("CREATE TABLE lang(name, first_appeared)")
# This is the named style used with executemany():
data = (
    {"name": "C", "year": 1972},
    {"name": "Fortran", "year": 1957},
    {"name": "Python", "year": 1991},
    {"name": "Go", "year": 2009},
)
cur.executemany("INSERT INTO lang VALUES(:name, :year)", data)
# This is the qmark style used in a SELECT query:
params = (1991,)
cur.execute("SELECT * FROM lang WHERE first_appeared = ?", params)
print(cur.fetchall())
```

CSV

CSV (Comma-Separated Values) es un formato de archivo utilizado para representar datos de forma tabular. En un archivo CSV, cada línea del archivo representa una fila de datos, y los valores dentro de esa fila están separados por comas o cualquier otro delimitador (los más comunes son comas y tabulador).

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Por lo general, la primera fila de un archivo CSV se utiliza para los encabezados, proporcionando nombres para cada columna.

Python cuenta con la librería **csv** para trabajar con este tipo de ficheros, con la cual se pueden especificar los delimitadores y los caracteres que rodean a cada valor, por lo que con la misma librería se pueden extraer y escribir valores en cualquier formato de archivos separados por un delimitador especial.

Trabajar con este tipo de ficheros tiene ventajas destacables, como:

- **Simplicidad y legibilidad:** El formato CSV es fácil de entender y leer, lo que lo hace accesible para usuarios y aplicaciones.
- **Compatibilidad universal:** Es compatible con una amplia gama de software y aplicaciones, incluidas hojas de cálculo, bases de datos y programas de análisis de datos.
- **Facilidad de creación y edición:** Puede crearse y editarse con herramientas simples, como editores de texto, o con software especializado para manipulación de datos.
- **Almacenamiento de datos tabulares:** Se utiliza comúnmente para representar datos en forma tabular, donde cada columna contiene un tipo específico de información.
- **Delimitadores personalizables:** Aunque se llama "Comma-Separated Values", los delimitadores no están limitados a comas. Pueden ser puntos y comas, tabulaciones u otros caracteres, según la necesidad.
- **Transferencia de datos:** CSV se utiliza comúnmente para transferir datos entre sistemas, ya que es un formato simple y ampliamente compatible.
- **Integración con software de análisis:** Muchas herramientas de análisis de datos, como Excel, R y Python (con bibliotecas como Pandas), admiten la importación y exportación de datos en formato CSV.
- **Almacenamiento intermedio:** Puede utilizarse como formato de almacenamiento intermedio entre sistemas o etapas de un proceso de análisis de datos.
- **Intercambio de datos en la web:** En el ámbito web, se utiliza a menudo para el intercambio de datos entre aplicaciones y servicios.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

- **Automatización de procesos:** Al ser un formato de texto plano, es fácil de manipular y automatizar mediante scripts o programas, facilitando la manipulación y transformación de datos.

Aunque el formato CSV es simple y ampliamente utilizado, tiene limitaciones, como la falta de soporte para tipos de datos complejos o estructuras anidadas. En situaciones donde se requiere mayor complejidad en la representación de datos, se pueden utilizar formatos más avanzados como JSON o XML. Sin embargo, para muchos escenarios prácticos y para datos tabulares simples, CSV sigue siendo una opción eficaz y fácil de utilizar en el manejo de datos.

Para acceder desde Python a este tipo de ficheros, se puede realizar mediante la librería csv directamente:

```
import csv

def lectura_csv(nombre_fichero):

  with open(nombre_fichero, 'r') as fr:

    reader = csv.reader(fr)

    for fila in reader:

      print(fila)

lectura_csv('mis_datos.csv')
```

Se puede posteriormente crear una clase de tipo **dataclass** o un **namedtuple** y añadir tipado o alguna funcionalidad extra.

Sin embargo, existe otra opción para leer y escribir ficheros con delimitadores, y es mediante la librería Pandas (Pandas, 2023), la cual provee las funciones **read_csv** y **write_csv**:

```
import pandas as pd

df = pd.read_csv('mis_datos.csv')
```

La ventaja de utilizar pandas es que los tipos de las columnas pueden ser inferidos y el tratamiento de datos complejos, como fechas, puede ser especificado. Inclusive se pueden importar archivos mediante URL:

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
url = "https://ejemplo.com/datos.csv"

df = pd.read_csv(url)
```

Y al ser un objeto de tipo DataFrame, se pueden utilizar métodos útiles para exploración, por ejemplo, **df.head()** para revisar los primeros elementos, o **df.info()** para un resumen con la información más importante. Además de los métodos para manipulación y análisis de datos, revisados en el contenido de la anterior semana, e incluso el método **df.to_csv()** para exportar estos datos manipulados nuevamente a un formato CSV.

XML

Los archivos CSV son especialmente útiles cuando se pretende guardar datos que comparten la misma estructura o que no tengan jerarquías definidas. Cuando se quiere guardar datos más complejos y con jerarquías, se usan otros tipos de formatos. Uno de los más populares es XML (eXtensive Markup Language). Este formato permite definir jerarquías en los datos de manera independiente respecto al sistema en el que se estén utilizando y tienen dos componentes principales: las etiquetas y los atributos. Las etiquetas forman los bloques de datos y pueden contener (o no) atributos. Los ficheros XML se utilizan para guardar información, pero también para ser intercambiados entre diferentes sistemas de bases de datos o aplicaciones.

Python también provee herramientas de manejo de este tipo de ficheros. La librería estándar para trabajar con ficheros XML es **xml.etree.ElementTree** y se suele importar como **ET** para abreviar. Un ejemplo con las funcionalidades básicas se presenta a continuación:

```
import xml.etree.ElementTree as ET

# Cargar el archivo XML en un objeto ElementTree

tree = ET.parse('nombre_del_archivo.xml')

# Obtener la raíz del árbol XML

root = tree.getroot()

# Ejemplo de iteración sobre elementos hijos

for child in root:
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
    print(child.tag, child.attrib)
# Ejemplo de búsqueda de un elemento específico
elemento_especifico = root.find('nombre_del_elemento')
print(elemento_especifico.text)
# Modificar un elemento existente
elemento_especifico.text = 'Nuevo texto'
# Crear un nuevo elemento y agregarlo al árbol
nuevo_elemento = ET.Element('nuevo_elemento')
nuevo_elemento.text = 'Contenido del nuevo elemento'
root.append(nuevo_elemento)
# Guardar el árbol modificado en un nuevo archivo XML
tree.write('nuevo_archivo.xml')
```

JSON

El formato JSON (JavaScript Object Notation) es también ampliamente conocido y utilizado en la industria de las aplicaciones web, ya que permite crear parseadores de contenido de forma rápida, e intercambiar datos entre aplicaciones utilizando un número de caracteres extra muy pequeño. Este formato se suele comparar con XML, debido a que ambos se utilizan para intercambiar datos. Sin embargo, JSON necesita menos caracteres para enviar la misma información y, por tanto, se utiliza más a menudo, principalmente en aplicaciones web que usan AJAX.

Su sintaxis es simple y solo puede contener los siguientes elementos: números (positivos, negativos, enteros, decimales y notación científica), cadenas de caracteres o strings (de longitud variable, definidos entre comillas dobles), booleanos, vacio o null (equivalente al None), listas (arrays de longitud variable) y objetos (pares de clave-valor separados por dos puntos).

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Desde luego, Python cuenta con la librería **json** que da soporte a los documentos en este formato, y se presenta un ejemplo, con las funcionalidades básicas a continuación:

```
import json
# Cargar el contenido de un archivo JSON
with open('nombre_del_archivo.json', 'r') as archivo_json:
    datos_json = json.load(archivo_json)
# Ejemplo de acceso a un valor específico
valor_especifico = datos_json['clave']
print(f"Valor específico: {valor_especifico}")
# Ejemplo de modificación de un valor
datos_json['clave'] = 'Nuevo valor'
# Crear un nuevo diccionario JSON
nuevo_json = {'nueva_clave': 'nuevo_valor'}
# Modificar el objeto original con el nuevo diccionario
datos_json.update(nuevo_json)
# Guardar los datos modificados en un nuevo archivo JSON
with open('nuevo_archivo.json', 'w') as nuevo_archivo_json:
    json.dump(datos_json, nuevo_archivo_json, indent=2)
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Tablib y Pandas

Existen librerías de terceros que permiten trabajar con ficheros en diferentes formatos y así simplificar las interfaces para acceder, modificar y guardar esos ficheros. Las dos librerías más importantes (librerías de terceros) que permiten trabajar con ficheros en diferentes formatos directamente para simplificar interfaces de acceso, modificación y guardado son Tablib y Pandas. Ambas soportan formatos como Excel, JSON, YAML, DataFrames, HTML, Jira, TSV, CSV, SQL, HDF5, docx, mp3, mp4, entre otros. Estas librerías convierten dichos formatos en DataFrames (y viceversa), así que se pueden realizar operaciones sobre los datos de forma agregada fácilmente. La principal diferencia entre Tablib y Pandas es que Tablib es liviana y está orientada a la interacción con ficheros en diferentes formatos, mientras que Pandas hace esta interacción de forma lateral (como una funcionalidad extra), dado que su principal cometido es la manipulación científica de grandes volúmenes de datos (Jiménez, 2021).

Conclusiones

Los ficheros son herramientas fundamentales para el almacenamiento permanente de información en el ámbito de las aplicaciones informáticas. Estos conjuntos de datos relacionados entre sí se guardan en dispositivos de almacenamiento duradero, como discos duros o memorias flash, y pueden ser tratados como unidades independientes. La característica distintiva de la permanencia radica en la capacidad de los datos de persistir incluso después de que el programa que los creó ha dejado de ejecutarse, a menos que ocurran fallos remarcables o hasta que sean eliminados intencionalmente.

La manipulación de datos estructurados y no estructurados es facilitada por las librerías de Python. Una de las ventajas de usar Python para interactuar con bases de datos es la amplia gama de librerías disponibles, lo que facilita el proceso. Además, Python es un lenguaje de programación muy utilizado y cuenta con una gran comunidad de desarrolladores que comparten sus habilidades y desarrollan constantemente nuevas herramientas y librerías.

SQLite3 en Python proporciona una interfaz simple y eficaz para interactuar con bases de datos SQLite. Al ser una base de datos incorporada, es ideal para aplicaciones más pequeñas o situaciones donde se requiere una solución liviana y sin necesidad de un servidor de bases de datos independiente. La

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

combinación de la facilidad de uso de Python y la capacidad de SQLite hace que sea una opción popular para proyectos que requieren una base de datos rápida y sin complicaciones.

Aunque existen librerías específicas para cada formato de archivos, como: csv xml.etree.ElementTree o json, entre otros, existe la opción también de trabajar mediante librerías de terceros que permiten trabajar con ficheros en diferentes formatos y así simplificar las interfaces para acceder, modificar y guardar. Estas librerías son Tablib y Pandas, en donde ambas soportan formatos como Excel, JSON, YAML, DataFrames, HTML, Jira, TSV, CSV, SQL, HDF5, docx, mp3, mp4, entre otros.

Referencias

Jiménez, Ó. R. (2021). *Python a fondo*. Marcombo.

Pandas. (2023, octubre 26). *pandas – Python Data Analysis Library*. https://pandas.pydata.org/

Trespaderne, F. M., & Echevarría, R. M. (2023). *Ficheros – Fundamentos de Programación en Python*. https://www2.eii.uva.es/fund_inf/python/notebooks/10_Ficheros/Ficheros.html

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Autor:
Ronie Martínez

Persistencia en bases de datos

Introducción

En esta lectura de profundización se examinarán en detalle los tipos de bases de datos más populares y utilizados en la industria, así como sus conexiones para manipularlas con Python. Una de las mejores formas de almacenar datos organizados es utilizando una base de datos. Además, tienen la característica de que los datos pueden consultarse y cambiarse rápidamente y de manera efectiva.

Se han desarrollado varios tipos de bases de datos. Cada individuo se esfuerza por abordar un problema específico de manera más efectiva que sus competidores, lo que resulta en la existencia de bases de datos adaptadas a todos los gustos y aplicaciones. Es responsabilidad del diseñador de sistemas informáticos comprender las diferencias y evaluar las diversas soluciones para encontrar la herramienta que más se adapte a sus necesidades.

En sistemas de bases de datos se pueden definir dos grandes bloques o grupos diferenciados: las bases de datos SQL y las NoSQL. Cada grupo contiene una gran cantidad de bases de datos que exceden los contenidos de este curso. Sin embargo, se muestran las características principales de cada grupo, algunos ejemplos de ellos, la finalidad de cada tipo de base de datos y cómo interactuar con ellas a partir de Python.

La utilización de una base de datos suele realizarse de forma externa (está guardada en su propio servidor, en su propio contenedor, etc.), lo que permite que muchas aplicaciones diferentes se puedan conectar a la misma, a diferencia de la persistencia en ficheros locales (en la máquina donde se ejecuta el código). Esto también permite el escalamiento horizontal (con múltiples instancias) y vertical (con hardware mejorado).

Es así como se han creado industrias enteras relacionadas con las bases de datos y los productos externos, lo que permite la creación de una variedad de plataformas de servicios de bases de datos. En la actualidad, una gran cantidad de estas plataformas se encuentran en la nube (en servidores externos alrededor

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

de todo el mundo), como Oracle Cloud, Amazon Web Services, Microsoft Azure, IBM Cloud y Google Cloud.

Cada plataforma ofrece una variedad de servicios, incluidas bases de datos y otros tipos (monitorización, contenedores, guardado de información, correo electrónico, etc.). Además, debido a que muchas desarrollan sus propios sistemas, la oferta de servicios para conectar una aplicación es muy amplia y toma tiempo conocer todas las posibilidades.

Con el objetivo de tener una API (Application Programming Interface) común para conectarse a bases de datos desde Python se creó la PEP-249 (Lemburg, 1999), la cual recoge métodos y características que deberían tener los conectores de cualquier base de datos. Dado que muchos de los conectores para diferentes bases de datos usan los mismos métodos, cada uno con algunas funciones extra que dependerán de la base de datos donde se pretenda conectar, en esta lectura de profundización se detallarán las funciones más comunes y su utilización. Esta lectura está basada en parte del Capítulo 7 del libro "Python a Fondo" (Jiménez, 2021).

Bases de datos

Funciones básicas para todos los conectores

La función básica para realizar una conexión a la base de datos es connect y se define como se presenta a continuación:

- **connect(dsn='host:db_name', user='user', password='pass'):** esta función sirve para crear una conexión a la base de datos definida en el parámetro dsn usando como usuario y contraseña los parámetros específicos. Esta función puede tener características particulares dependiendo del conector y la base de datos que se vayan a utilizar, pero siempre debe devolver un objeto tipo Connection.
    - Para la definición de dsn se suele utilizar el siguiente esquema: esquema-o-driver://usuario:contraseña@hostname: puerto.

Las funciones más importantes que deben implementar todas las instancias de la clase **Connection** son las siguientes:

- **Connection.close():** Cierra la conexión abierta con la base de datos.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

- **Connection.commit():** Envía todos los cambios pendientes de realizar en la conexión desde que se abrió. Es común que esta operación no sea necesaria si la conexión permite hacer un commit automático con cada cambio, pero depende de las opciones y la configuración de cada conexión y de cada conector de base de datos.
- **Connection.rollback():** Permite deshacer los cambios que aún no se hayan guardado en la base de datos. Es la operación opuesta a commit y solo es efectiva si no se ha configurado un commit automático. Este método es opcional pero muy utilizado.
- **Connection.cursor():** Devuelve una instancia de la clase Cursor, que es la encargada de realizar operaciones con la base de datos a la que el cursor está enlazado.

Las operaciones en las bases de datos se ejecutan utilizando cursores, que son instancias de la clase Cursor. Implementan principalmente las siguientes funciones:

- **Cursor.callproc(nombre_procedimiento, *args):** Permite ejecutar llamadas a procedimientos guardados en las bases de datos. Este método es opcional.
- **Cursor.close():** Cierra el cursor asociado. Se recomienda que siempre que se abra un cursor, también se cierre, para no provocar múltiples conexiones abiertas o bloqueos de tablas accedidas por diferentes cursores sin cerrar. Una vez cerrado un cursor, este no podrá ser utilizado, sino que debe crearse uno nuevo.
- **Cursor.execute(sentencia, *args):** Permite ejecutar cualquier sentencia soportada (o no) por la base de datos dentro del cursor asociado.
- **Cursor.executemany(sentencia,secuencia_de_parametros):** Permite ejecutar la misma sentencia para un iterador de parámetros. Esta sentencia se utiliza cuando se quieren hacer inserciones en serie de un iterador de parámetros a modo de filas.
- **Cursor.fetchone():** Permite recibir el primer valor devuelto por la sentencia ejecutada en el cursor.
- **Cursor.fetchmany(size=cursor.arraysize):** Permite recibir una cantidad seleccionable de resultados sobre el cursor asociado.
- **Cursor.fetchall():** Devuelve todos los resultados sobre el cursor asociado.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

- **Cursor.arraysize:** es un atributo que define la cantidad de líneas que debe leer el cursor en cada iteración, por defecto es 1.

Capa de abstracción de bases de datos en Python

Existe una librería que pretende hacer más fácil la integración de Python con cualquier base de datos creando una mínima capa de abstracción. Esta librería llamada pyDAL (Di Pierro et al., 2014/2019) permite las conexiones con bases de datos, la definición de tablas, la inserción de elementos, las uniones de tablas, las búsquedas agregadas y las búsquedas básicas o anidadas sin que sean dependientes de cada conector; además, soporta las siguientes bases de datos: SQLite, MySQL, PostgreSQL, MSSQL, FireBird, Oracle, DB2, Ingres, Sybase, Informix, Teradata, Cubrid, SAPDB, IMAP y MongoDB. Esta librería no suele venir por defecto en las distribuciones de Python, por lo que es necesario instalarla mediante el siguiente comando:

pip install pyDAL

from pydal import DAL, Field

Dentro de los métodos más utilizados están:

- **db_main = DAL('sqlite://ejemplo.db'):** Para crear la base de datos de nombre ejemplo.
- **db.define_table():** Para definir una tabla.
- **db.tabla_n.drop():** Para eliminar la tabla de nombre tabla_n.
- **db.tabla_n.truncate():** Para truncar la tabla de nombre tabla_n, es decir, quita todas las filas de una tabla o las particiones especificadas de una tabla, sin registrar las eliminaciones individuales de filas.
- **db.tabla_n.insert():** Para agregar un registro a la tabla de nombre tabla_n.
- **db_main(query).select():** Para hacer una consulta en la base de datos db_main.

Se recomienda usar esta librería para hacer operaciones básicas sobre las bases de datos soportadas. Para extraer todo el potencial de cada base de datos es necesario conocer y realizar operaciones específicas de cada base de datos y, para ello, se recomienda buscar un conector de Python específico para cada caso.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Librería para consultas SQL en crudo

Existe una librería que permite interactuar con muchas bases de datos relacionales diferentes mediante consultas SQL directamente. Esta librería se llama records (Reitz, 2014/2023) y se puede instalar de forma simple con el siguiente comando:

```
pip install records[pandas]
```

```
import records
```

Esta librería usa SQLAlchemy (ORM) y tablib para realizar las consultas y formatear el resultado, lo que la convierte en una herramienta muy simple, pero a la vez muy poderosa. Dentro de sus características están: capacidad de lanzar consultas parametrizadas en SQL puro, securización de parámetros para evitar inyecciones de SQL, posibilidad de ejecutar ficheros de código SQL, detección automática de las columnas que serán devueltas por la consulta, operaciones simples para detectar la información de la base de datos, posibilidad de mostrar el contenido de las filas en forma de diccionario, posibilidad de exportar los datos consultados en múltiples formatos como JSON, YAML, DataFrame, XLS, CSV, etc., soporte para transacciones, cacheado de filas en las consultas para mejorar el rendimiento de revisiones futuras y capacidad de realizar operaciones en bloque (bulk operations).

Un ejemplo de la utilización de esta librería se presenta a continuación:

```
db = records.Database('sqlite:///ejemplo.db')

tablas = db.get_table_names()

print(f'Tablas disponibles: {tablas}')

usuarios = db.query('SELECT * FROM usuarios')

print(f'El primer usuario es: {usuarios[0]}')
```

Las consultas realizadas se pueden exportar en diversos formatos, por ejemplo, CSV o JSON:

```
for formato_ex in ('csv', 'json'):

  with open('usuarios.' + formato_ex, 'w') as fw:

    fw.write(usuarios.export(formato_ex))
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Esta librería es muy simple, dado que su principal función es permitir realizar consultas SQL de cualquier tipo. No obstante, también se trata de una herramienta muy potente, ya que permite exportar los datos o pasar parámetros directamente desde Python de forma intuitiva.

SQLAlchemy - ORM

Para trabajar con objetos en Python y guardarlos en una base de datos (SQL o NoSQL), generalmente se busca eliminar la mayor complejidad posible para poder crear código que no tenga que lidiar con consultas en bruto a la base de datos, serialización y deserialización de objetos, consultas específicas para cada base de datos o migraciones cuando hay cambios en las tablas o la lógica de la aplicación. Los ORM (Object Relational Mapping) se crearon para abordar este problema y son una herramienta esencial para trabajar con diferentes tipos de objetos, cuando los objetos tienen relaciones entre sí y cuando se pretenden realizar operaciones que resultarían en una gran cantidad de consultas a la base.

La mayoría de los ORM soportan la definición de los campos que debe tener cada tabla como si fueran atributos de una clase en Python y, además, ayudan a manejar los cambios que se pueden realizar en esas clases con una herramienta llamada "migraciones". El ORM crea y valida la estructura de la base de datos cuando se crean las clases en Python, y crea migraciones de la base de datos cuando se producen cambios, lo que hace que todo el proceso sea mucho más mantenible.

SQLAlchemy es un ORM de Python destinado a la industria y cumple con estándares de calidad similares a los de cualquier otro sistema comparable. Se presenta como una alternativa de código abierto para empresas que requieran una capa de abstracción de objetos para relacionar los datos guardados en la base de datos. Como no tiene dependencia externa, puede integrarse de forma aislada en diferentes proyectos. Por esta razón, es el método más común para usar con frameworks que no tienen su propio ORM para usar con bases de datos.

Para mantener el control de las consultas que ocurren en todo momento cuando se ejecuta el programa, uno de los objetivos principales de este marco es crear una fina capa de abstracción entre los objetos por relacionar y la base de datos. Si es necesario, siempre puede usar consultas SQL en bruto. Los siguientes ejemplos muestran que la sintaxis es totalmente declarativa.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Bases de datos NoSQL

Las bases de datos relacionales NoSQL rompen con algunas restricciones y paradigmas. La característica principal de las bases de datos NoSQL es que no están obligadas a cumplir con reglas establecidas o prácticas relacionales buenas, por lo que conceptos como transacciones o uniones entre datos quedan en un segundo plano para favorecer otros aspectos de la base de datos, como la velocidad de lectura o la opción de escalar horizontalmente más fácil y rápida. El usar un sistema de bases de datos NoSQL no significa que no haya soporte para realizar operaciones SQL, sino que no existen las restricciones ACID.

Dada esta flexibilidad, existen muchas opciones que se centran en un área específica de la computación, desde bases de datos orientadas a guardar solo elementos como clave-valor hasta bases de datos que guardan los datos de forma columnar, pasando por algunas que utilizan nodos y grafos para guardarlos.

Las bases de datos para aplicaciones en tiempo real, como videojuegos, aplicaciones médicas, aplicaciones de aviación, banca electrónica y precios de mercado financiero son otro tipo de bases de datos NoSQL. Los datos se leen y modifican constantemente en este tipo de bases de datos, lo que requiere una gran cantidad de conexiones, una respuesta rápida y la capacidad de actualizar una parte aislada de la base de datos, así como la capacidad de replicar los cambios en todos los sistemas que leen esa información.

Conclusiones

Esta lectura de profundización se enfocó en la persistencia de datos en las bases de datos, específicamente en los tipos de bases de datos más populares y utilizados en la industria, así como en sus conexiones para manipularlas con Python de manera efectiva. Las bases de datos ofrecen una excelente manera de almacenar datos organizados, permitiendo consultas y modificaciones rápidas y eficientes.

La diversidad en el desarrollo de bases de datos ha resultado en soluciones adaptadas a una variedad de necesidades y aplicaciones. La responsabilidad recae en el diseñador de sistemas informáticos para comprender estas diferencias y evaluar las opciones disponibles, encontrando la herramienta más adecuada para sus propósitos. La evolución de esta tecnología ha dado lugar a industrias enteras centradas en bases de datos y servicios externos. La mayoría

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

de estas plataformas, en la actualidad, se encuentran en la nube, ofreciendo una variedad de servicios, desde bases de datos hasta monitorización.

Se presentaron algunas librerías, entre las que está pyDal cuyo objetivo es hacer más fácil la integración de Python con cualquier base de datos creando una mínima capa de abstracción. Esta librería permite las conexiones con bases de datos, la definición de tablas, la inserción de elementos, las uniones de tablas, las búsquedas agregadas y las búsquedas básicas o anidadas sin que sean dependientes de cada conector. Una característica importante es que soporta gran variedad de bases de datos, entre las que están: SQLite, MySQL, PostgreSQL, MSSQL, FireBird, Oracle, DB2, Ingres, Sybase, Informix, Teradata, Cubrid, SAPDB, IMAP y MongoDB.

Otra de las librerías presentadas fue records que permite interactuar con muchas bases de datos relacionales diferentes mediante consultas SQL directamente. También se listaron y detallaron funciones básicas para los conectores, necesarios para las conexiones a las bases de datos.

También se presentó un breve resumen de SQLAlchemy – ORM para trabajar con objetos en Python y guardarlos en una base de datos (SQL o NoSQL), cuyo objetivo es eliminar la complejidad. Y, finalmente, se presentó un breve resumen de bases de datos NoSQL, tipos y aplicaciones.

Referencias

Di Pierro, M., Barillari, G., Bizzotto, S., Valleri, P., Sweeny, N., Leuthi, M., Clark, C., clach05, Lengyel, D., & Rauch, S. (2019). *pyDAL* [Python]. web2py. https://github.com/web2py/pydal (Obra original publicada en 2014)

Jiménez, Ó. R. (2021). *Python a fondo*. Marcombo.

Lemburg, M.-A. (1999). *PEP 249 – Python Database API Specification v2.0 | peps.python.org*. https://peps.python.org/pep-0249/

Reitz, K. (2023). *Records: SQL for Humans™* [Python]. https://github.com/kennethreitz/records (Obra original publicada en 2014)

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

+34 691 225 633
Rev. Febrero 2024

Autor:
Ronie Martínez

Herramientas para manipulación y análisis de datos

Introducción

En la era digital actual, la cantidad de datos generados diariamente es asombrosa. Desde transacciones financieras hasta interacciones en redes sociales, cada acción en línea deja una huella digital. En este contexto, la habilidad para manipular y analizar datos se ha convertido en un activo invaluable para profesionales en diversos campos. En el corazón de esta capacidad se encuentran conceptos fundamentales como bases de datos, lenguajes de consulta (SQL y NoSQL), APIs, webservices y scripting.

Palabras clave

Bases de datos, SQL y NoSQL, API, webservices, scripting.

Reto

Debido al crecimiento exponencial de los datos, es importante explorar diversas herramientas, marcos de trabajo y plataformas que nos permitan realizar manipulación y análisis sobre estos. En este tema se abordarán varios de estos conceptos fundamentales mediante definiciones y comparaciones.

Desarrollo

En la ciencia de datos y big data, la manipulación de datos se puede utilizar de diversas maneras. Los datos deben organizarse para que las máquinas puedan leerlos e interpretarlos, mientras que, por otro lado, es necesario enseñar a las personas a utilizarlos. Dado que la cantidad de datos consumidos y almacenados aumenta exponencialmente, esto es más crucial que nunca.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Todas las industrias tienen un impacto directo en el funcionamiento de las organizaciones debido a la capacidad de adquirir conocimiento. Para que las computadoras funcionen correctamente, los datos estructurados, o los datos a los que solo pueden acceder las computadoras, deben seguir un patrón.

Después de que los datos hayan sido procesados y modificados para proporcionar información útil, los humanos deben limpiar y mapear los datos, todo esto, a medida que se procesan y utilizan más datos.

El proceso de organizar los datos para hacerlos más comprensibles se conoce como manipulación de datos. Para facilitar la comprensión, cualquier tipo de datos puede ordenarse siguiendo algún criterio de conveniencia (por ejemplo, alfabéticamente, por categorías, etc.). Puede ser difícil encontrar a una persona específica en una empresa si la información de los empleados no está organizada. Se puede ordenar alfabéticamente toda la información de los empleados, lo que facilita el acceso a la información individual de cada empleado. Esto permite a los propietarios de sitios web hacer un seguimiento del tráfico y de los sitios más populares, y los registros del servidor web suelen utilizarlo.

Esta técnica es utilizada por los usuarios de contabilidad para ordenar los datos con el fin de determinar los costos de producción, las futuras obligaciones fiscales, las tendencias de precios, etc. Ayuda a los expertos en previsión bursátil a predecir el rendimiento de las acciones en el futuro. Además, puede usarse para mostrar datos de manera más realista en formatos de datos, código de software o páginas web. Los programas informáticos, las páginas web o el formato de datos determinado por el usuario pueden modificar los datos para que sean más fáciles de entender (Ortega, 2023).

Bases de datos

Las bases de datos desempeñan un papel crítico y un pilar fundamental en la manipulación y almacenamiento de datos, proporcionando un marco estructurado y eficiente para almacenar, gestionar y acceder a grandes volúmenes de datos. La comprensión profunda de las bases de datos relacionales, respaldadas por el lenguaje SQL, ofrecen una estructura organizada y tabular, además, permite a los profesionales diseñar esquemas eficientes, normalizar datos, realizar análisis avanzados, ejecutar consultas complejas, descubrir patrones, entrenar modelos de aprendizaje automático y extraer información valiosa para la toma de decisiones. Esta semana, exploraremos cómo optimizar el rendimiento de las consultas, relacionar e integrar diversos conjuntos de datos,

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

implementar transacciones seguras y diseñar esquemas que se adapten a las necesidades específicas de cada aplicación.

Además, nos sumergiremos en las bases de datos NoSQL, explorando su flexibilidad para manejar datos no estructurados. Analizaremos casos de uso específicos para bases de datos de documentos, grafos y clave-valor, proporcionando a los estudiantes una perspectiva completa de las opciones disponibles en el panorama de almacenamiento de datos.

Lenguajes de consulta (SQL y NoSQL)

El lenguaje de consulta SQL (Structured Query Language) es un lenguaje estándar diseñado para gestionar y manipular bases de datos relacionales. Se utiliza para definir y manipular datos en sistemas de gestión de bases de datos relacionales (Relational Database Management System – RDBMS), como MySQL, PostgreSQL, SQL Server y Oracle. Permite realizar operaciones como consultas, inserciones, actualizaciones y eliminaciones de datos de manera eficiente.

Como características principales se tienen:

- **Tablas Relacionales:** Los datos se almacenan en tablas relacionadas entre sí. Las relaciones entre entidades de datos pueden ser eficientemente establecidas usando claves foráneas (foreign keys), habilitando consultas sofisticadas y asegurando la coherencia de los datos.
- **Esquemas Predefinidos:** Al seguir un modelo de datos estructurado, se define la estructura de los datos antes de insertarlos. Lo que a su vez ayuda a la integración, gestión y organización de contenidos en plataformas CMS (Content Management Systems).
- **Transacciones:** Permite operaciones atómicas, consistentes, aisladas y duraderas (Atomicity, Consistency, Isolation, Durability – ACID). Garantiza la consistencia de los datos y previene la pérdida y corrupción de datos.
- **Ecosistema maduro:** Al haber sido utilizadas durante décadas, se tiene como resultado un ecosistema robusto en cuanto a herramientas, marcos de trabajo (frameworks) y documentación extensa.

Por otro lado, se tienen las bases de datos NoSQL o "Not Only SQL" que es un término que engloba diversas tecnologías de bases de datos que no siguen el modelo relacional tradicional. No impone esquemas fijos y es más flexible en

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

el manejo de datos no estructurados o semiestructurados, como MongoDB o Cassandra, que ofrecen enfoques alternativos para casos donde la flexibilidad y escalabilidad son prioritarias. Sin embargo, la integridad de los datos es algo a considerar, ya que se pueden requerir comprobaciones adicionales a nivel de aplicación.

Como características principales se tienen:

- **Esquemas Dinámicos:** No requiere una estructura de datos predefinida. Permite a los desarrolladores adaptarse a los requisitos de datos cambiantes sin la necesidad de migraciones complejas.
- **Escalabilidad Horizontal:** Se puede escalar fácilmente añadiendo más nodos al sistema y no necesariamente añadiendo más recursos en un solo lugar. Esto permite manejar volúmenes de datos masivos para aplicaciones de crecimiento impredecible.
- **Tipos de Datos Variados:** Admite diferentes tipos de datos, como documentos (JSON o BSON), clave-valor, grafos, etc.
- **Diseño:** Prioriza consistencia, disponibilidad o tolerancia de partición, basado en el teorema CAP (Consistency, Availability, or Partition Tolerance).

Las diferencias más importantes se presentan en la Tabla 1, y hay que considerar que la elección entre SQL y NoSQL depende de los requisitos específicos del proyecto, la complejidad de los datos y las necesidades de escalabilidad (CodeAutomation, 2023; MongoDB, 2023; Smallcombe, 2023).

TABLA 1. COMPARACIÓN ENTRE SQL Y NOSQL.

	SQL	NoSQL
Modelo de datos	Datos organizados en tablas relacionadas.	Datos almacenados en formatos flexibles como documentos, grafos o clave-valor.
Escalabilidad	Escalabilidad vertical (más recursos en una única máquina).	Escalabilidad horizontal (añadir más máquinas).
Esquemas	Esquema fijo y riguroso.	Esquema dinámico o inexistente.
Almacenamiento	Los datos son almacenados en tablas con filas y columnas	Los datos son almacenados en colecciones o documentos.
Transacciones	ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad).	Puede variar; no todos cumplen con ACID.
Ejemplos de aplicación	Sistemas donde la integridad de los datos es crítica (por ejemplo, sistemas financieros).	Aplicaciones que requieren escalabilidad y flexibilidad, en donde no se requieran datos estructurados (por ejemplo, entornos de redes sociales, Big Data, IoT, etc.)
Gestores	MySQL, PostgreSQL, Oracle, SQL Server, Microsoft SQL Server.	MongoDB, Cassandra, Couchbase, Amazon DynamoDB, Redis.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Entre los factores a tomar en cuenta para seleccionar el sistema de base de datos apropiado están:

- **Estructura de datos:** Analizar el modelo de datos y determinar si se beneficia de una estructura rígida (SQL) o requiere la flexibilidad de datos sin estructura (NoSQL).
- **Necesidades de escalabilidad:** Evaluar el potencial de crecimiento de la aplicación y evaluar si el escalado horizontal (NoSQL) o el escalado vertical (SQL) se alinea mejor con los requisitos a largo plazo.
- **Consistencia vs. flexibilidad:** Identificar la importancia de la consistencia de los datos y si la aplicación puede tolerar cierta flexibilidad en estos.
- **Comunidad y soporte:** Considerar la madurez del sistema de base de datos, la documentación disponible y el tamaño de la comunidad de desarrolladores que lo rodea.

APIs y Servicios Web

La interconexión de sistemas se logra a través de APIs (Application Programming Interfaces) y servicios web. Estos mecanismos permiten la transferencia de datos entre aplicaciones de manera segura y eficiente. Comprender cómo consumir y proporcionar datos a través de APIs se ha vuelto esencial en el entorno tecnológico actual.

Una API es un conjunto de reglas y definiciones que permite que una aplicación se comunique con otra. Proporciona un conjunto de funciones y procedimientos que permiten la creación de aplicaciones y facilitan la interacción entre diferentes componentes de software.

Un cliente realiza una solicitud HTTP (HyperText Transfer Protocol) a una API en algún punto al que tiene acceso. La API valida la solicitud y la pasa al servidor o aplicación de destino. El servidor devuelve una respuesta, primero a través de la API, quien luego la transfiere al cliente. La API proporciona seguridad al desacoplar la aplicación consumidora de la infraestructura que proporciona el servicio. Una vez que un servicio API y un cliente intercambian claves de autenticación, se comunican a través de la API, lo que significa que no tienen acceso completo entre sí, pero se puede pasar suficiente información de forma segura para realizar el trabajo. Un cliente puede emitir ciertos tipos de solicitudes de protocolo HTTP, de los cuales existen cuatro tipos principales:

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

- **GET:** esta solicitud lee información de un registro en la base de datos.
- **PUT:** esta solicitud cambia la información de un registro en la base de datos.
- **POST:** esta solicitud crea un nuevo registro en la base de datos.
- **ELIMINAR:** esta solicitud elimina un registro de la base de datos.

Técnicamente, lo anterior describe el uso de API web, que es otra razón por la que se mezclan las API y los servicios web. Sin embargo, las APIs pueden existir fuera de la web y lo han hecho desde que las computadoras se conectaron entre sí por primera vez. La distinción importante es que las API se utilizan para vincular aplicaciones, y aunque hoy en día eso generalmente se hace a través de HTTP o HTTPS, no siempre fue así.

Como características principales, se tienen:

- **Interoperabilidad:** Permite a aplicaciones de diferentes proveedores interactuar entre sí.
- **Abstracción de Funcionalidades:** Oculta la complejidad interna y expone solo las funciones necesarias.
- **Documentación:** Incluye información detallada sobre cómo utilizar la API.

Un webservice es un conjunto de protocolos y estándares que permite la comunicación entre sistemas y máquinas a través de la web. Utiliza protocolos como HTTP para facilitar la transmisión de datos y se basa en estándares como XML (eXtensible Markup Language) o JSON (JavaScript Object Notation) para estructurar la información que se intercambia entre las aplicaciones.

Como características principales, se tiene:

- **Acceso Remoto:** Permite el acceso a funciones o datos en un servidor remoto.
- **Estándares Abiertos:** Utiliza estándares como SOAP (Simple Object Access Protocol) o REST (REpresentational State Transfer).
- **Escalabilidad:** Puede manejar grandes volúmenes de tráfico y peticiones.

Una comparación se presenta en la Tabla 2, y como resumen, mientras que una API es un conjunto general de reglas para la interacción entre aplicaciones, un webservice es una implementación específica de estas reglas que permite la comunicación entre sistemas a través de la web, utilizando protocolos y estándares específicos.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

TABLA 2. COMPARACIÓN ENTRE API Y WEBSERVICE

	API	Webservice
Propósito	Define cómo las aplicaciones deben interactuar.	Facilita la comunicación entre sistemas a través de la web.
Protocolos	Puede utilizar diversos protocolos, incluidos HTTP, RPC (Remote Procedure Call), etc.	Utiliza protocolos específicos como SOAP o REST.
Formato de datos	Puede usar diversos formatos de datos, como JSON o XML.	Frecuentemente utiliza XML para estructurar los datos (aunque RESTful webservices también pueden usar JSON).
Uso típico	Puede referirse a cualquier interfaz de programación, incluyendo bibliotecas de funciones en un lenguaje específico.	Se refiere específicamente a servicios web que siguen protocolos y estándares.
Ejemplos prácticos	Google Maps API, Twitter API.	RESTful Web Service para acceder a datos de usuarios (GET, POST, PUT, DELETE), SOAP service para integración empresarial.

Cada servicio web es una API ya que expone los datos y/o la funcionalidad de una aplicación, pero no toda API es un servicio web. Las APIs son solo los terminales que permiten las relaciones cliente-servidor, mientras que los servicios web son un estilo arquitectónico para integrar aplicaciones web que utilizan los estándares abiertos XML, SOAP y WSDL (Web Services Description Language) a través de una red troncal de protocolo de Internet. XML es el formato de datos utilizado para contener los datos y proporcionar metadatos a su alrededor, SOAP se utiliza para transferir los datos y WSDL se utiliza para describir los servicios disponibles. Es muy común que las API y los servicios web se confundan ya que tienen mucho que ver entre sí (Jagger, 2023).

Scripting para manipulación de datos

El scripting para la manipulación de datos es un proceso esencial para cambiar o alterar datos con el fin de hacerlos más legibles, organizados y útiles. Este proceso puede incluir operaciones como filtrado de filas y columnas, agregación, combinación, concatenación, entre otros. Los lenguajes de scripting son ampliamente utilizados en la manipulación de datos, ya que están diseñados para realizar tareas específicas y automatizar procesos en entornos determinados, como: la creación y el mantenimiento de sitios web, la generación de contenido dinámico, la interacción con bases de datos, la

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

automatización de tareas en sistemas operativos, el desarrollo de aplicaciones móviles y de escritorio, etc.

El uso de lenguajes de scripting, como Python o R, facilita la manipulación, transformación y análisis eficiente de grandes conjuntos de datos. Estos lenguajes son herramientas poderosas para la automatización de tareas repetitivas, creación de scripts personalizados y análisis estadístico, lo que permite ahorrar tiempo y reducir errores, en comparación con enfoques manuales.

Dentro de sus características principales están:

- **Automatización de tareas:** Los scripts facilitan la automatización de tareas repetitivas, como limpieza de datos, conversión de formatos o generación de informes.
- **Manipulación eficiente de datos:** Permite la manipulación eficiente de grandes conjuntos de datos, aplicando operaciones específicas a través de instrucciones codificadas.
- **Análisis Exploratorio de Datos (EDA):** Facilita el EDA al proporcionar herramientas para visualización, estadísticas descriptivas y otras técnicas de análisis exploratorio.
- **Integración con herramientas y librerías específicas:** Se integra fácilmente con librerías especializadas para análisis de datos, como Pandas en Python o tidyverse en R.
- **Flexibilidad y adaptabilidad:** La flexibilidad de los lenguajes de scripting permite adaptarse a diversos tipos de datos y escenarios de manipulación.

Una comparación entre los lenguajes más utilizados, Python y R, se presenta en la Tabla 3, y la elección para scripting en manipulación de datos depende de varios factores, incluyendo preferencias personales, requisitos del proyecto, y el contexto en el que se está trabajando. Ambos lenguajes son muy populares en el ámbito de la ciencia de datos y manipulación de datos.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

TABLA 3. COMPARACIÓN ENTRE PYTHON Y R

	Python	R
Utilización	Ampliamente utilizado para scripting en manipulación y análisis de datos, con librerías como Pandas y NumPy.	Especialmente popular en estadísticas y análisis de datos, con paquetes como tidyverse.
Sintaxis y enfoque	Sintaxis fácil de aprender, enfoque generalista con amplia aplicabilidad en diversos dominios.	Diseñado específicamente para análisis estadístico, con una sintaxis centrada en manipulación y visualización de datos.
Librerías	Amplio ecosistema de librerías, no solo para manipulación de datos, sino también para aprendizaje automático, visualización y más.	Fuerte enfoque en paquetes especializados para estadísticas y análisis de datos.

Análisis de datos en tiempo real

A medida que se incrementa la cantidad de datos, se requiere un sistema capaz de manejarlos rápidamente. El análisis de datos en tiempo real recopila, procesa y clasifica datos sin apenas retrasos, todo esto con el fin de tomar decisiones de inmediato y elaborar informes que saquen conclusiones a partir de la información.

El análisis de datos en tiempo real puede usarse para una variedad de propósitos, incluida la detección de patrones, incidencias o personalización de la información, detectar incidencias, optimiza los recursos y personalizar la información en tiempo real, ayudando a tomar decisiones ágiles.

Existen varias herramientas y plataformas que pueden ayudar en este proceso (Arias, 2023):

- **Apache Kafka:** Es una plataforma distribuida de streaming de eventos de alto rendimiento y baja latencia que facilita la construcción de arquitecturas de microservicios y el análisis en tiempo real de los flujos de datos.
- **Esper:** Es una plataforma especializada en el Procesamiento de Eventos Complejos (CEP) que permite el procesamiento en tiempo real de eventos y datos, así como el análisis de patrones, anomalías y correlaciones en flujos de datos en tiempo real.
- **Redis:** Es una base de datos en memoria preparada para procesar la información y recuperar la data de manera instantánea y eficiente.
- **TensorFlow Serving:** Es una plataforma de alto rendimiento y flexible que permite introducir el aprendizaje automático en el análisis de datos en

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

tiempo real (entornos de producción), para conseguir una toma de decisiones inteligente e instantánea.

- **Arquitecturas Event-Driven (EDA):** Son sistemas que trabajan con eventos (acciones o cambios de estado significativos) en tiempo real para analizar de forma instantánea la información.

Conclusiones

La manipulación de datos puede utilizarse en la ciencia de datos de diversas maneras. Se utiliza para hacer los datos más comprensibles o estructurados. Los datos se utilizan mejor cuando pueden manipularse con fines de marketing, ventas, contabilidad y atención al cliente. Un análisis de datos adecuado implica reordenar, ordenar, modificar y cambiar los datos.

Las bases de datos son la columna vertebral de los entornos de ciencia de datos y big data. Proporcionan la infraestructura necesaria para almacenar, gestionar y analizar datos a escala, permitiendo a las organizaciones extraer conocimientos valiosos y tomar decisiones informadas en un mundo cada vez más impulsado por datos. La elección entre SQL y NoSQL depende en gran medida de la naturaleza del proyecto y sus requisitos, las características y los requisitos específicos de los datos y las metas a largo plazo. Ambos enfoques ofrecen ventajas y desventajas significativas, y la decisión adecuada dependerá de factores como la estructura de los datos, la escalabilidad, la consistencia y la flexibilidad. Inclusive, en algunos casos, una combinación de ambos enfoques (tecnología políglota) puede ofrecer la mejor solución, aprovechando las fortalezas de cada sistema en diferentes partes del proyecto. La clave reside en entender las necesidades del negocio y de los datos para tomar decisiones informadas sobre la arquitectura de la base de datos.

APIs y webservices son conceptos interrelacionados pero distintos en el ámbito de la comunicación entre sistemas. Ambos juegan un papel crucial en la construcción de aplicaciones modernas y la integración de servicios, pero se diferencian en su alcance y enfoque. Mientras que la API representa un conjunto más amplio de reglas y definiciones que permiten la interacción entre diferentes aplicaciones o componentes de software; no se limita a la comunicación web y puede abarcar cualquier interfaz de programación, incluyendo librerías de funciones en un lenguaje específico; ofrece flexibilidad en la implementación, ya que puede utilizar varios protocolos de comunicación, como HTTP, RPC, entre otros; y puede tener una documentación detallada que

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

guía a los desarrolladores sobre cómo utilizar las funciones proporcionadas. Webservices por su parte es una implementación específica de una API que utiliza protocolos web estándar (como SOAP o REST), para facilitar la comunicación entre sistemas a través de la web; se enfoca específicamente en la comunicación remota mediante la exposición de funciones o servicios a través de URLs; puede seguir estándares abiertos y protocolos bien definidos, facilitando la interoperabilidad entre sistemas heterogéneos; y puede ser RESTful, lo que implica un diseño orientado a recursos y el uso de operaciones HTTP estándar. En última instancia, la elección entre API y webservices dependerá de los requisitos del proyecto, la arquitectura de la aplicación y la naturaleza de la comunicación entre sistemas. Ambos conceptos son esenciales en el desarrollo moderno de software, permitiendo una integración fluida y eficiente de servicios en el vasto ecosistema digital actual.

El scripting para manipulación de datos ofrece una herramienta poderosa para profesionales de datos al facilitar la automatización de tareas, la manipulación eficiente de datos y la exploración detallada de conjuntos de datos. La elección del lenguaje de scripting dependerá de las preferencias individuales y de los requisitos específicos del proyecto, pero en conjunto, estos scripts son esenciales para mejorar la eficiencia y la calidad en el manejo de datos.

Referencias

Arias, A. (2023, agosto 17). *Análisis de datos en tiempo real: Consulta esta guía*. Tokio School. https://www.tokioschool.com/noticias/analisis-datos-tiempo-real/

CodeAutomation. (2023). *Working with Databases in Web Development: SQL vs. NoSQL*. https://www.linkedin.com/pulse/working-databases-web-development-sql-vs-nosql-codeautomationai

Jagger, E. (2023). *What's the Difference Between API and Web Services?* Abstract. https://www.abstractapi.com/guides/api-vs-web-services

MongoDB. (2023). *NoSQL Vs SQL Databases*. MongoDB. https://www.mongodb.com/nosql-explained/nosql-vs-sql

Ortega, C. (2023, febrero 13). Manipulación de datos: Qué es, técnicas y ejemplos. *QuestionPro*. https://www.questionpro.com/blog/es/manipulacion-de-datos/

Smallcombe, M. (2023). *SQL vs NoSQL: 5 Critical Differences*. Integrate.Io. https://www.integrate.io/blog/the-sql-vs-nosql-difference/

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.